LeoWay - System Specification

8 October 2026 · Version 0.1 · Draft for review

1. Purpose and scope

LeoWay helps students at HTL Leonding plan their journey to school. It combines their real lesson times with public transport data, warns about delays and prepares delay appeals that teachers can check.

This specification describes what the system must do and how its main parts work together. It follows a compact HTL Pflichtenheft structure. LeoWay is a working name; the final brand name is still open. Technical values marked **proposed** are draft settings, not confirmed school requirements.

Initial situation and goals

Transport apps do not know when a student's first lesson changes. Students may notice a delay too late, and teachers currently receive photos or screenshots as evidence. LeoWay should help students leave at the right time and make delay appeals easier to check.

Users and access

User	Main tasks
Student	Connect WebUntis, save a route, receive alerts, report disruptions and prepare an appeal.
Teacher	Open a shared delay record. In the optional portal, handle appeals from assigned students.
Administrator	Manage data adapters, check system health and handle support. Routine access does not include private timetable or appeal content.

System boundary

The system reads WebUntis and transport feeds. Students send appeal emails from their own account; teachers decide whether to accept them. LeoWay proves what its data source reported about a connection, not that a student was on the vehicle.

The product backlog also includes crowd reports, prediction, school statistics, the way home and custom alert rules. The teacher portal is optional. Later extensions are other schools, an open delay dataset and indoor navigation.

Out of scope: writing absences or excuses to WebUntis, GPS tracking, storing home addresses and a native iPhone app. iPhone users use the website.

2. Functional requirements

“Must” means required for the feature to be accepted. Features follow the proposal's backlog order; this document does not set yearly releases. The teacher portal remains optional.

ID	Required behaviour
F01	The recorder must collect available departures and delays continuously, including during holidays. Each record must identify the source and collection time.
F02	Students must connect WebUntis with their own username and access key. The system must read lessons, cancellations, substitutions, exams and short lateness entries. It must not store absence reasons or notes.
F03	Students must save their departure stop, route, walking time and time buffer. The system must calculate when to leave for the first real lesson and update the alert when that lesson changes.
F04	The system must show delays and warn when the selected journey is expected to arrive late. It must suggest an earlier or alternative connection when the available data supports one.
F05	Alerts must use Discord first, then push and email through the same notification interface. Android must support a leave-by alarm; if permission is missing, it must explain this and offer a normal notification.
F06	The system must suggest delay records matching a lateness entry and the saved journey. The student must review the match. One action must prepare an editable email with journey details and a verification link.
F07	The system must group disruption reports for the same line, stop and time. Proposed: three different signed-in students within ten minutes trigger a crowd warning. This warning must stay separate from official delay evidence.
F08	Prediction must use recorded history to suggest a delay risk and personal buffer. School statistics must show connections that often arrive after lessons start, without naming students.
F09	The way-home view must use the actual last lesson. Custom auspex-lang rules must allow conditions such as a delay over five minutes before an exam. Invalid rules must show a clear error.
F10	If built, the teacher portal must let enrolled teachers verify, approve or decline appeals from assigned students and see connection reliability. Other teachers continue to receive email.

Main student flow

Connect WebUntis → save route → check first lesson → choose a journey → receive a leave-by alert → receive delay updates. If marked late: review a suggested delay match → open the email draft → check and send it.

3. Design and interfaces

Angular provides the student website and optional teacher portal. Capacitor wraps the same app for Android. Quarkus runs the backend, PostgreSQL stores data, and Python trains the prediction model.

Part	Responsibility
Source adapters	Read WebUntis, transport feeds and optional weather data. Convert each source into a shared format.
Recorder and services	Store observations; calculate journeys and alerts; match lateness entries; create appeals and statistics.
Notification service	Send the same alert through the selected channel. Prevent duplicate alerts for the same event.
Prediction service	Train on recorded history and return a risk estimate. Use a baseline when the model is unavailable.

Data flow

WebUntis + transport feeds → adapters → backend + recorder → PostgreSQL.
The backend serves the app and verification pages. The notification service sends alerts. Python reads delay history and provides prediction results.

External interfaces

WebUntis: access through each student's key. Proposed: refresh before the morning commute, every 15 minutes during it, and on manual request. Cache results and reduce requests after errors; confirm limits with school IT.

Transport: Linz AG live data; MVO and ÖBB timetables; VAO live regional data only if licensed. Without live data, show scheduled times and a clear “timetable only” label. Polling must stay within each provider's limit; shared connections use cached data.

App API: authenticated HTTPS endpoints for routes, lessons, journeys, reports and appeals. Students can access only their own private data. All channels use the same alert ID, message and journey reference.

Calculation and stored data

Leave-by: select a journey whose arrival time plus the walk to school and chosen buffer is no later than the first real lesson. Subtract the walk to the departure stop from its departure time. If no journey fits, show a warning instead of a safe arrival claim.

Delay records: keep line, trip, stop, service date, planned time, reported time or delay, source and observation time. Mark predictions as predictions; call a time “actual” only when the source confirms it. Store timestamps in UTC and display Europe/Vienna time.

Appeal matching: use the journey selected for that day. Proposed: suggest a match when its expected school arrival is within 15 minutes of the lateness end time. Never silently choose between uncertain matches; the student confirms the connection.

4. Quality, security and operation

The values below are proposed targets for the pilot and must be checked on the agreed test server. External feed freshness and message delivery remain visible separately from backend response time.

ID	Quality requirement and check
Q01	Expected pilot size: 100 registered students and 20 active users at once. At least 95% of cached app API requests must finish within two seconds under this load.
Q02	For an active monitored journey, evaluate each newly received delay within 60 seconds. Send at most one alert per channel for the same change. Measure channel delivery separately.
Q03	A failed source must not stop other sources. Show the last update time. Proposed: mark live observations older than five minutes as stale and never show them as current.
Q04	A student must be able to prepare an appeal in under one minute after opening a matched lateness entry. Check this with three students using test data.
Q05	Test the website on current Chrome, Firefox and Safari, including phone widths. Important actions must work with a keyboard, have labels and show readable error messages.
Q06	Each adapter must have its own tests. Changes must pass CI and a code review before deployment. Keep automated checks for timetable changes, matching and access control.

Access and verification

Encrypt WebUntis access keys at rest. Keep the encryption key outside the database and repository. Use HTTPS, check permissions on every private request and never write access keys or private school data to logs. Students must be able to disconnect WebUntis and delete their account.

Verification links must use an unguessable token. Proposed: expire after 30 days, with a student option to revoke earlier. The page shows only the selected transport record, its source and collection time; it contains no student identity or timetable. Changes to the link must not allow editing the stored evidence.

The optional teacher portal must use school-approved login and checked teacher-to-class assignments. Its login method must be agreed before development. A shared verification link does not grant access to private appeals.

Retention and recovery

Keep saved routes and contact details until account deletion. Replace the timetable cache on sync. Keep appeals and matched lateness entries until the end of the school year. Delete account-linked active data on account deletion; retain public transport history separately.

Proposed: encrypted daily backups, a seven-day backup cycle and one restore test before the pilot. Deleted accounts must stay deleted after a restore. Agree hosting, privacy information, consent handling and operational responsibility with the school before using real student data.

5. Acceptance and delivery

The supervising teacher reviews the agreed scope with the team. Each delivered feature is accepted when its linked requirements and tests pass. Later backlog features do not block acceptance of an agreed earlier increment.

Test	Expected result
A01 · F01, Q03	Use a recorded feed, then disconnect it. Observations retain source and time; the app marks stale data and other adapters continue.
A02 · F02-F03	Cancel the first lesson in a test timetable. The next real lesson is used and the leave-by alert is recalculated. Check a daylight-saving change too.
A03 · F04-F05, Q02	Inject a delay that causes late arrival. The system evaluates it within 60 seconds and creates one alert per channel. Test an alternative route and denied Android alarm permission.
A04 · F06, Q04	Match a lateness entry to a saved journey and prepare a draft in under one minute. The link shows the same source record. Changed, expired and revoked tokens are rejected.
A05 · Security	Student A cannot read student B's routes or appeals. Account deletion removes active private data. A backup restore preserves that deletion.
A06 · F07-F10	Repeated reports from one account count once. Test prediction on later dates excluded from training; compare mean delay error against “same delay as yesterday” and “average delay of this trip”. If it loses, keep the stronger baseline. Check aggregate statistics, way-home changes and valid/invalid rules; test teacher permissions if the portal is built.
A07 · Q01, Q05-Q06	Run the pilot load test, browser checks and CI suite. Record results and complete a backup restore on the chosen server.

Deliverables and review

Deliver source code, database migrations, automated tests, deployment instructions, a short student guide and test results. Work stays in two-week Scrum sprints. Changes to scope or proposed settings are recorded in the backlog and reflected in the next specification version.

Decisions still open

Confirm the final name, product owner and hosting; school permission and WebUntis limits; VAO access; teacher email source and appeal format; optional teacher login. Until email lookup is agreed, the student enters and checks the form teacher's address. Review the proposed thresholds with the teacher before the pilot.

Basis and references

Scope: LeoWay - Project Proposal, supplied PDF, 8 October 2026. Structure: the HTL Leonding LeoCode system specification and the IsyFact specification template. These guide the structure; they do not establish certified compliance with an Austrian norm. A teacher-provided template takes priority. Web references checked on 8 October 2026.